EEE 6778. Applied Machine Learning II

Module 5. Multimodal Architectures

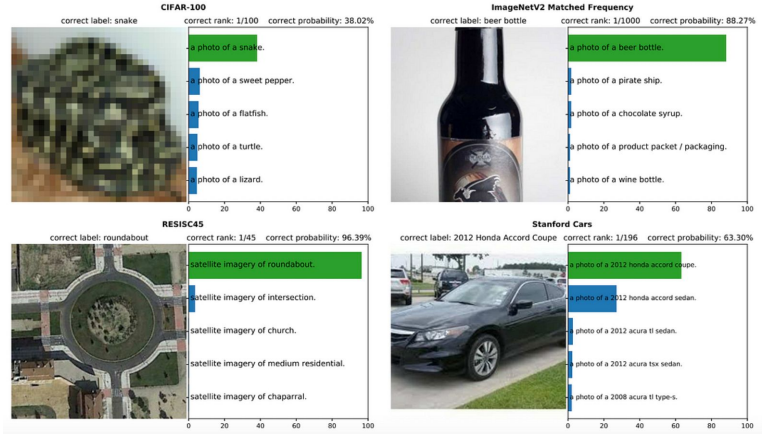
CLIP - Contrastive Language-Image Pretraining

Flamingo

CLIP

Introduction

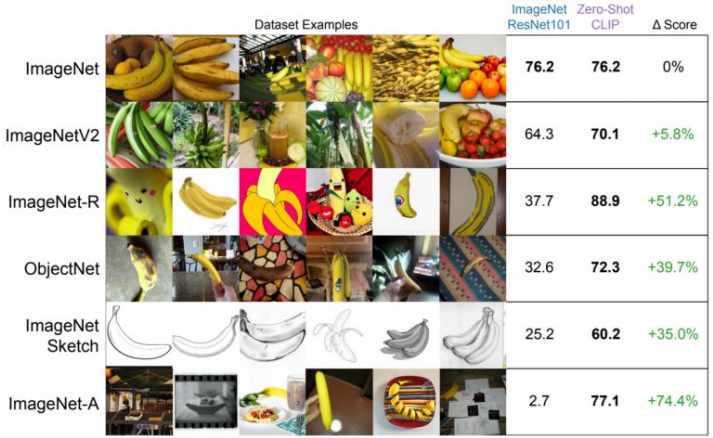
- CLIP stands for Contrastive Language–Image Pre-training
- Developed to create robust and generalizable image and language representations
- Enables zero-shot classification—no fine-tuning required on downstream tasks
- Useful for: Computer Vision, NLP, and Multimodal Applications



(Radford et al., 2021)

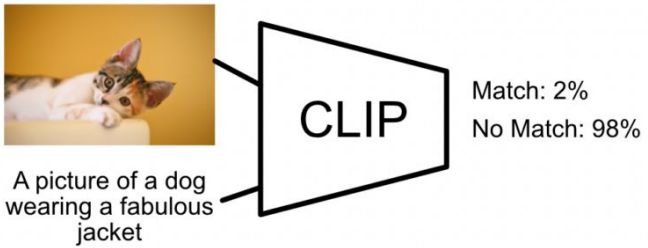
Traditional vs. CLIP-Based Classification

- Traditional models use supervised and semi-supervised learning with labeled datasets
- Struggle to generalize to new data distributions
- CLIP avoids overfitting by learning associations between images and captions Learns to recognize semantics, not just classes

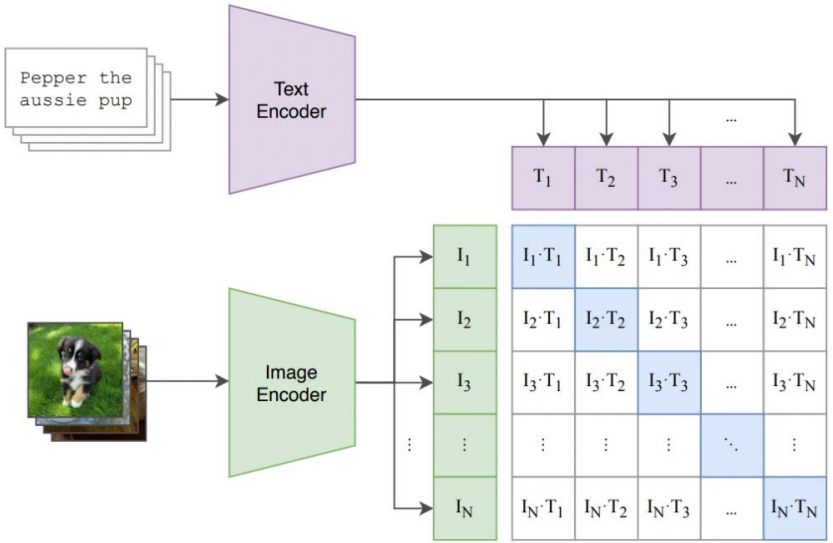


Core Concept – Contrastive Learning

- Learns by comparing positive (image-text) pairs vs. negative pairs
- Images and text are embedded into the same vector space
- Goal: Similar items = close, dissimilar = far in embedding space



(Warfield, 2023)

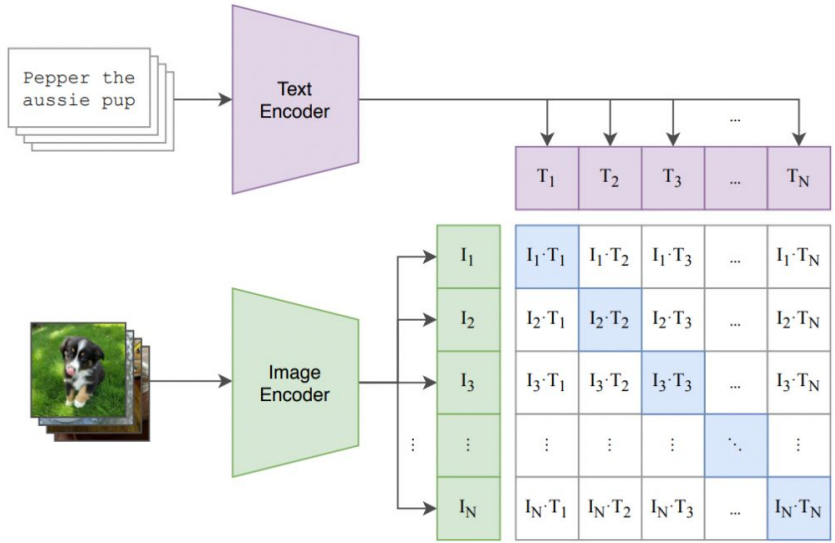


What CLIP Can Do

- Zero-shot Image Classification: Match an image with the best caption from a list
- Image Search: Retrieve the image most related to input text
- Use as Feature Extractor: Image/Text encoders used in downstream ML tasks
- Produces rich embeddings that capture deeper meaning

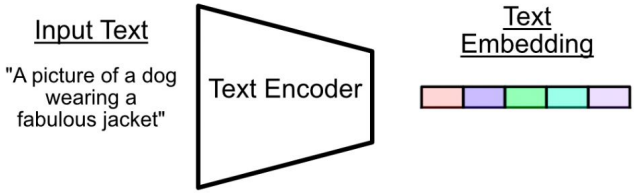
CLIP Architecture Overview

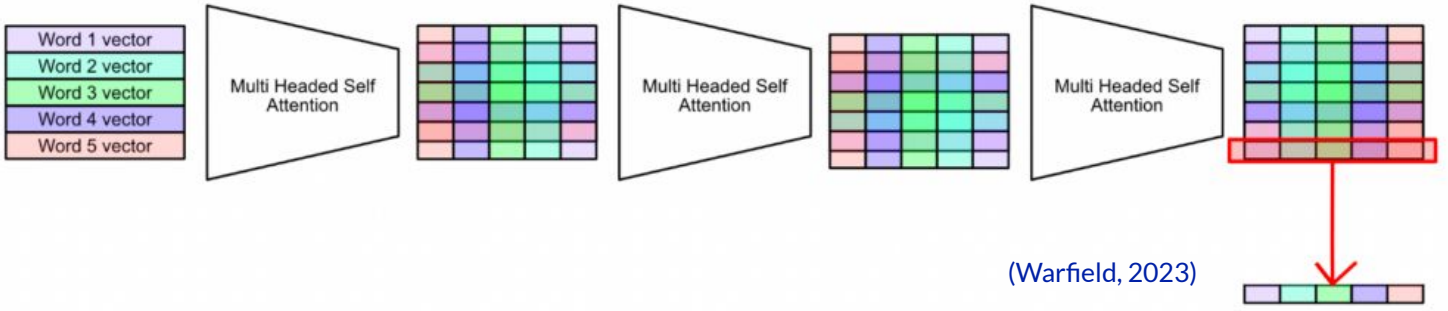
- Two main components: Text Encoder (Transformer)
- Image Encoder (CNN like ResNet or ViT)
- Outputs are projected into a joint multimodal embedding space



Text Encoder

- Based on a Transformer Encoder
- Uses self-attention to capture contextualized meanings
- Extracts the final token's vector to represent the whole input
- Outputs a single vector for each input sentence

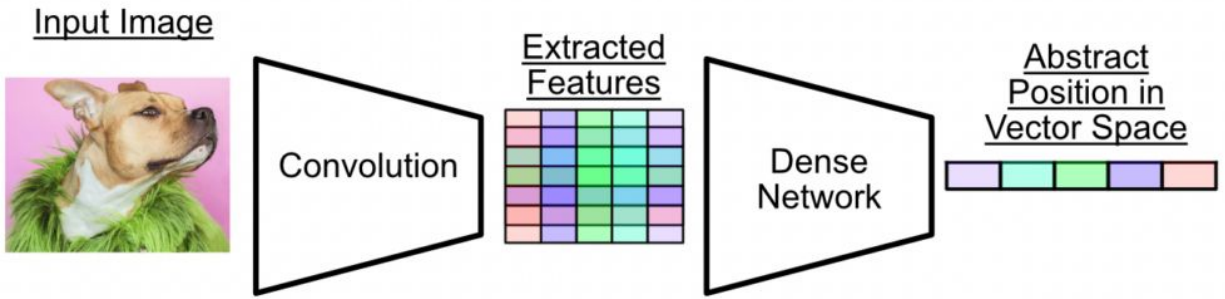




(Warfield, 2023)

Image Encoder

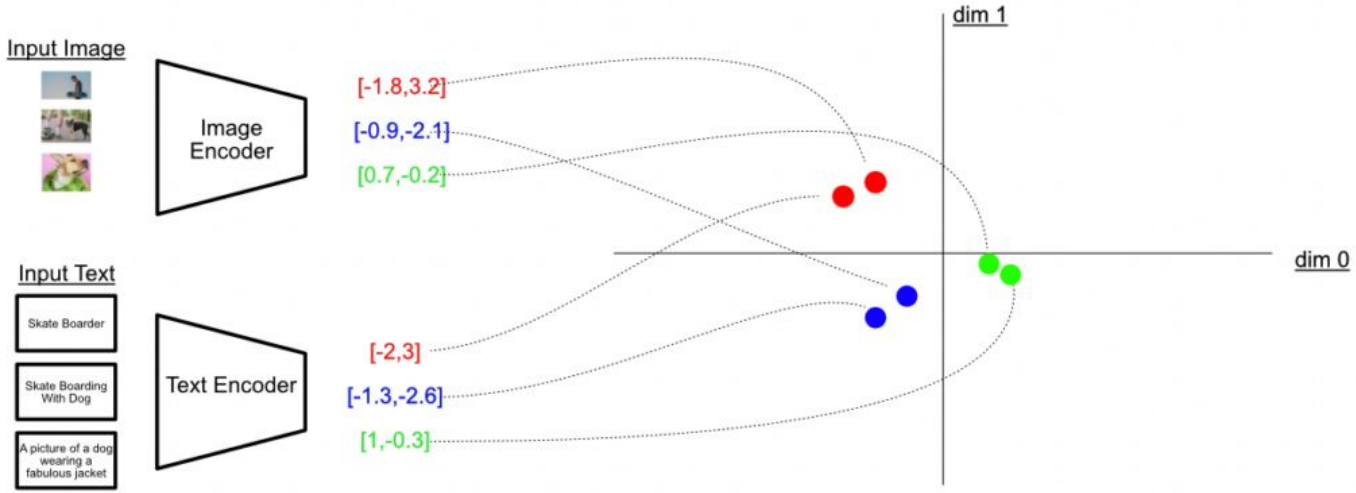
- Typically uses ResNet-50 or Vision Transformer
- Applies convolutions and pooling to capture spatial hierarchies
- Ends with a projection head (fully connected layer) to output a vector
- Final output: abstract summary of image content



(Warfield, 2023)

Multi-Modal Embedding Space

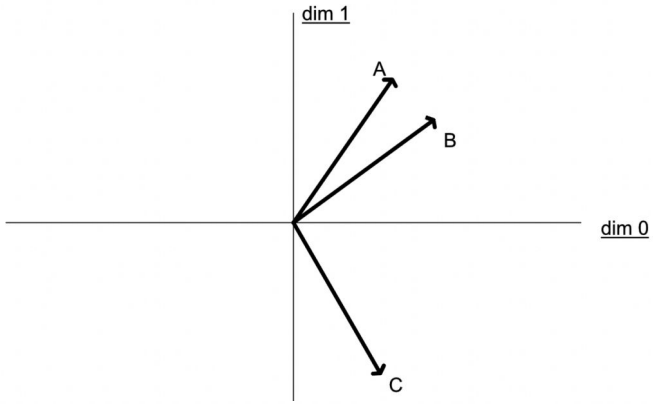
- Joint vector space where both images and texts are embedded
- Uses cosine similarity to measure "closeness" between image-text pairs
- Positive pairs should have high similarity, negatives should be low

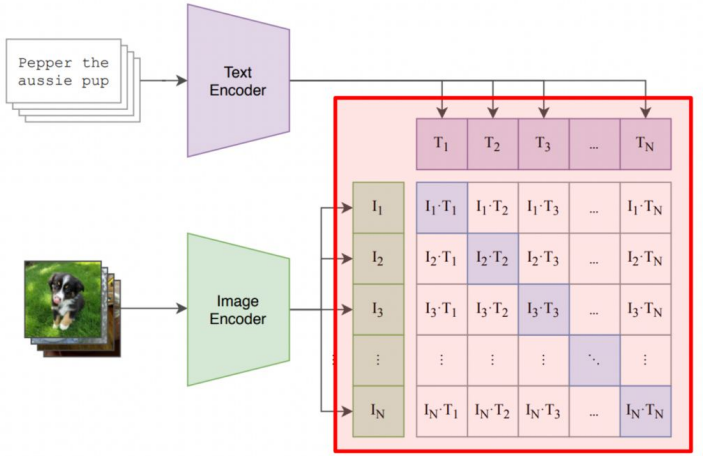


Cosine Similarity Refresher

- Measures the angle between two vectors
- Cosine of 0° = 1 (very similar), 90° = 0 (orthogonal), 180° = -1 (opposite)
- In CLIP, embeddings are normalized to unit vectors, simplifying computation

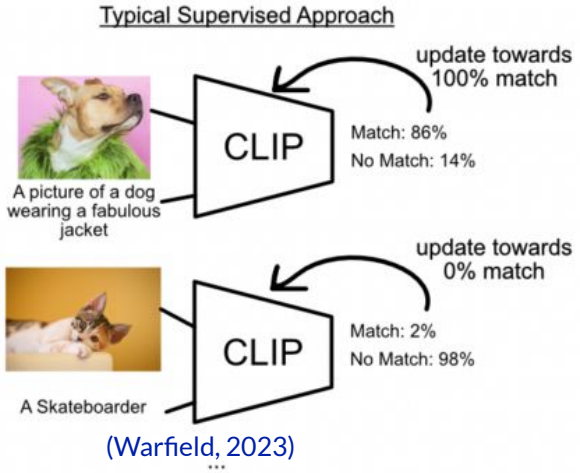
$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$



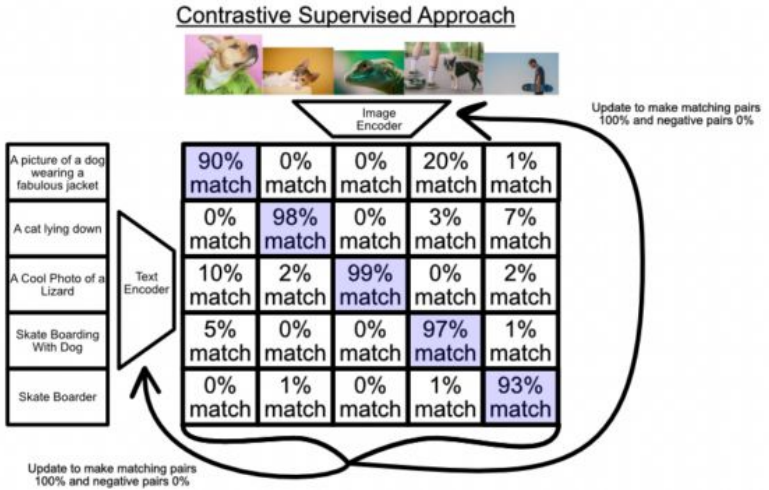


Contrastive Loss in CLIP

- Computes dot products (cosine similarity) between all image-text pairs
- Applies softmax across similarities to convert to probabilities
- Uses cross-entropy loss to maximize similarity for matching pairs
- Simultaneously pushes non-matching pairs apart

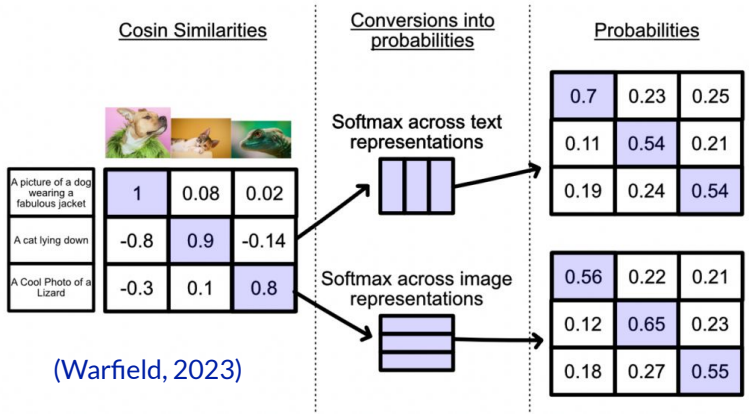


(Warfield, 2023)



Training Strategy

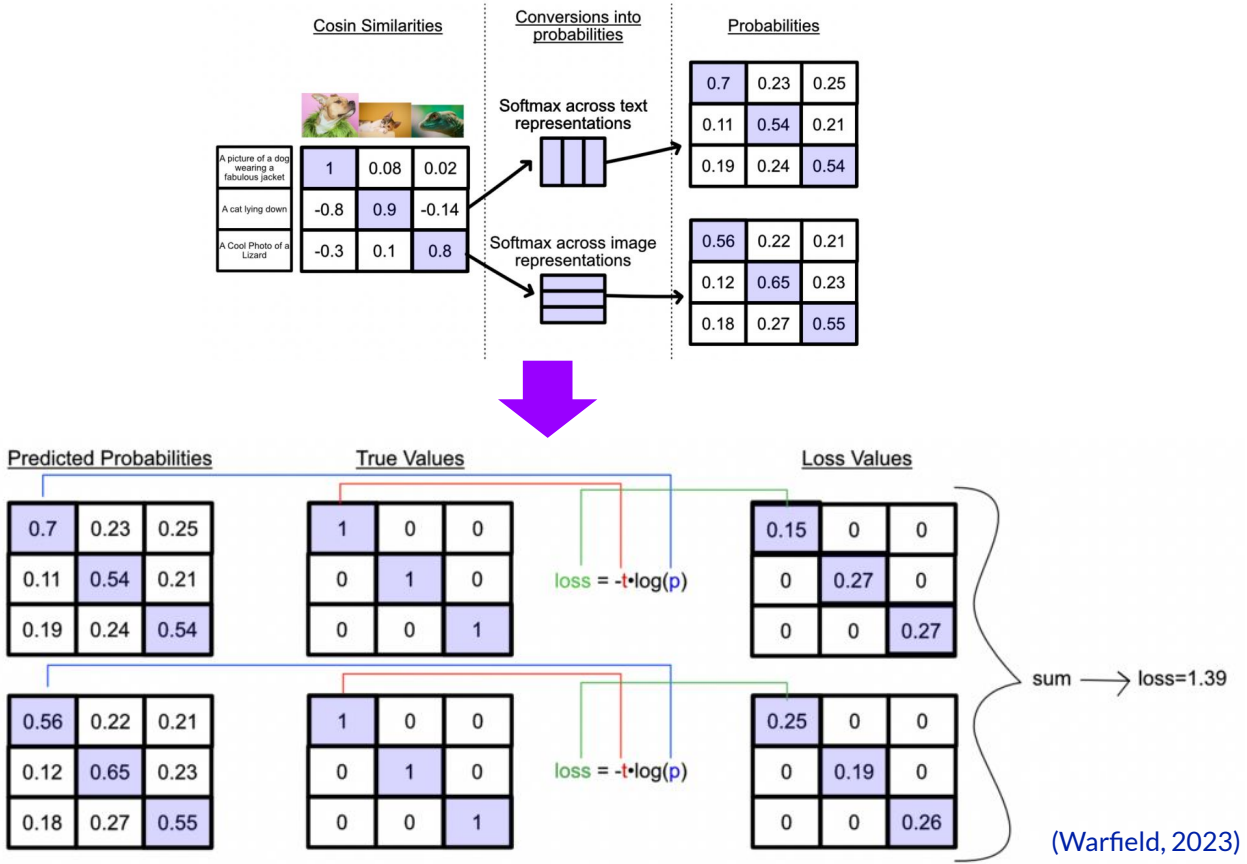
- Trained with large batch sizes (e.g., 32,768 image-text pairs)
- Encourages better generalization and robustness
- Uses symmetric loss: computes loss from both image-to-text and text-to-image



(Warfield, 2023)

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - temperature parameter

# 1) get a batch of aligned images and text
I, T = get_mini_batch()

# 2) extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# 3) joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# 4) scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# 5) symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Training Strategy



(Warfield, 2023)

Availability

- Open-sourced by OpenAI: The original CLIP model and code were released in 2021.
  - The repository provides: Pretrained models (ViT-B/32, RN50, etc.)
  - PyTorch implementation
  - Example scripts for zero-shot classification and feature extraction
- **Things to Keep in Mind**
  - Models are large and may need GPU for real-time inference or fine-tuning (if allowed).
  - CLIP is mostly used in a frozen (pretrained) state — meaning you use the encoders as-is without retraining.
  - Training from scratch (like OpenAI did) requires massive datasets (400M+ image-text pairs), but fine-tuning on custom tasks is doable.
- Original article here

FLAMINGO

Introduction to Flamingo

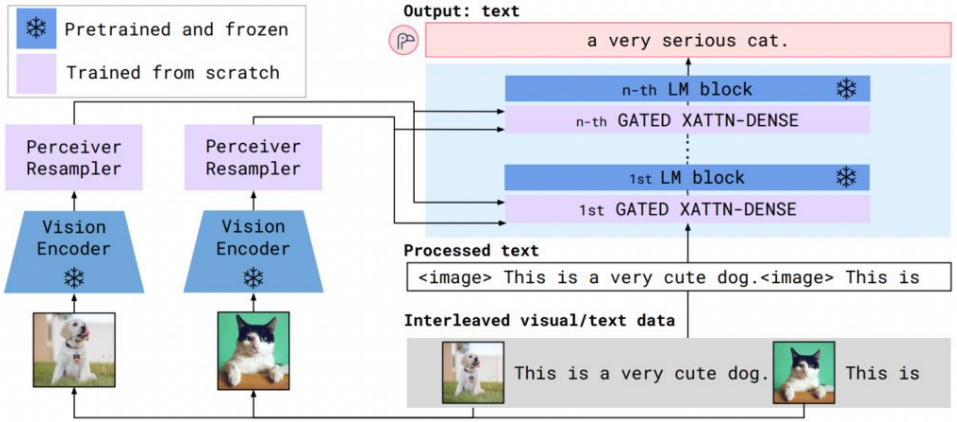
- Flamingo: a multimodal model combining vision and language understanding.
- Developed by DeepMind for few-shot learning with text and images.
- Builds on prior models like CLIP and decoder-only transformers.
- Supports tasks like visual Q&A, captioning, and image-text conversation.

Predecessors to Flamingo

- CLIP: aligns image and text in a shared vector space using contrastive learning.
- Decoder-only transformers: used in GPT-like language models.
- Flamingo combines:
  - CLIP's strong image representations (conceptually speaking)
  - Transformers' strong text generation ability

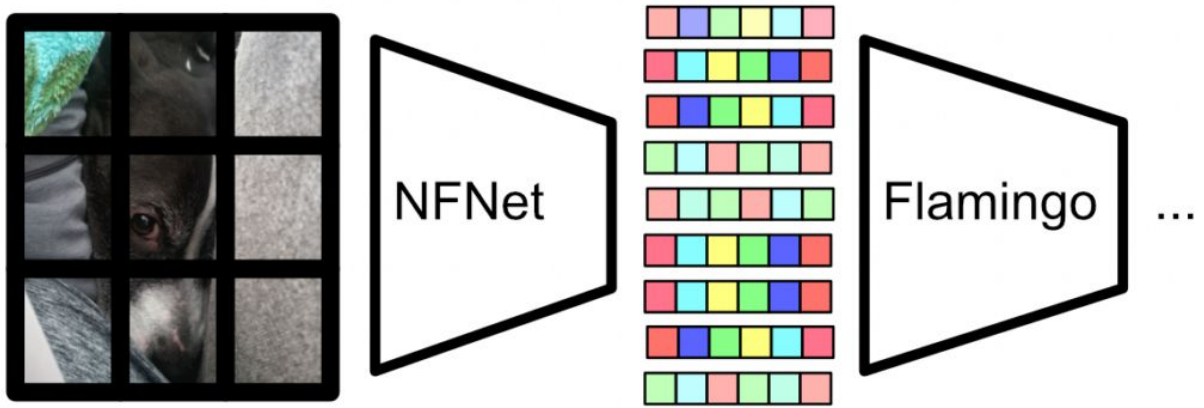
Flamingo in a Nutshell

- Four key components:
  - **Vision Encoder** (e.g., NFNet) to summarize image content
  - **Perceiver Resampler** to handle variable-length image inputs
  - **Gated Cross Attention** to blend visual data into text generation
  - **Pretrained Language Model** (like Llama or GPT)



Vision Encoder

- Uses a pretrained model (NFNet) to extract features from images.
- Converts image regions into dense, abstract representations.
- Enables the language model to understand visual content without retraining on images.

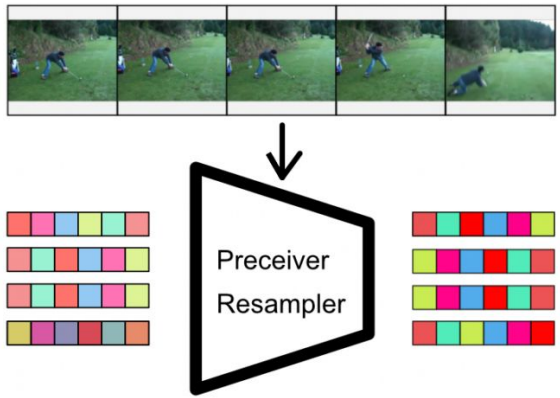


Perceiver Resampler

- Compresses a sequence of images into a fixed number of feature tokens.
- Supports both images and videos (treated as image sequences).
- Adds temporal encodings to maintain order in video data.
- Uses attention + feedforward layers with skip connections for efficient encoding.

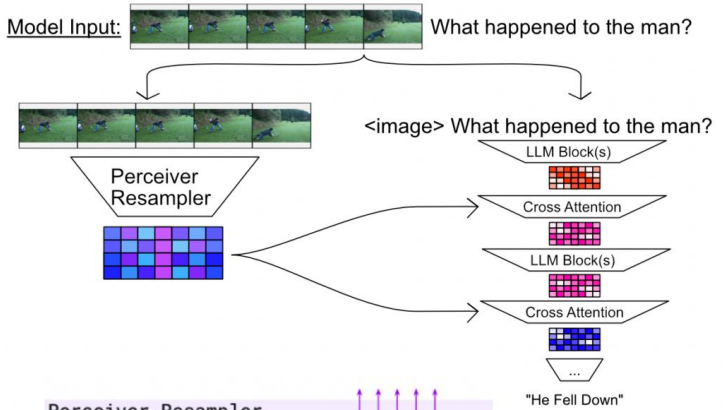
Perceiver Resampler

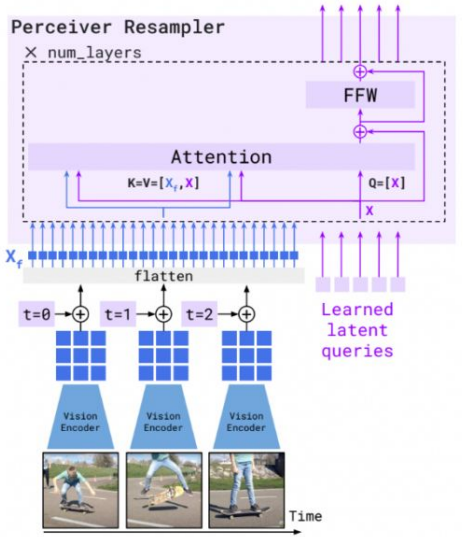
- Conceptually, you can think of the perceiver resampler as a filter; it takes in a fixed length of predefined tokens and uses input images extracted from video to filter those tokens.
- Regardless of the number of images in an input, the same fixed number of tokens come out of the output.



Perceiver Resampler

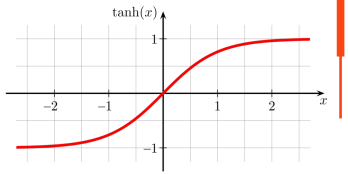
- From a high level, the perceiver resampler fits into the greater Flamingo architecture in the following way:
  - The images are extracted from the prompt.
  - In their place, an <image> token is placed in the text so that the model knows where the image came from.
  - The output from the perceiver resampler is used to incrementally filter the internal state of the LLM throughout various layers, ultimately allowing the LLM to converse about the images.

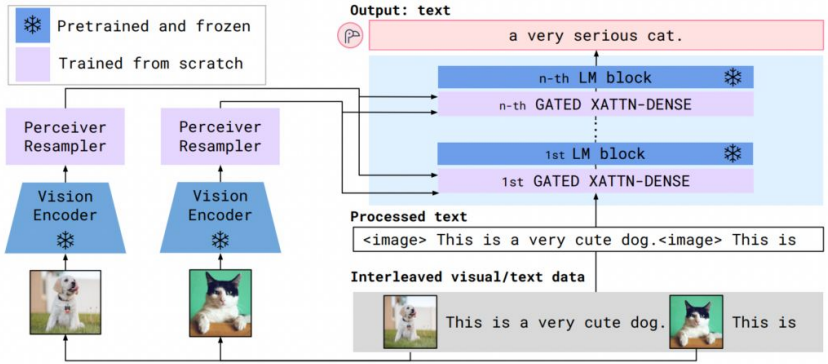


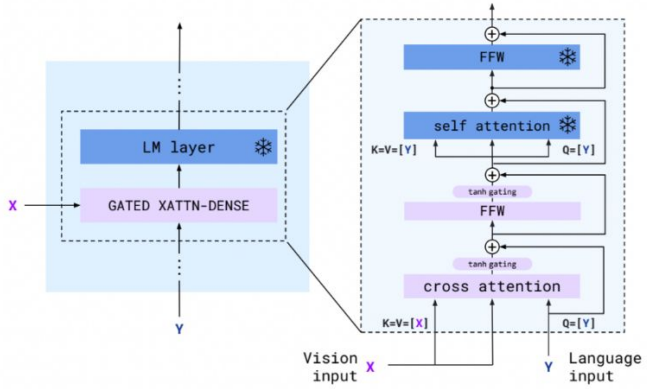


Gated Cross Attention

- Injects image features into the language model incrementally.
- Applies a tanh gate to control how much image information is used.
- Learns to "converse" about images in context with text.







How Flamingo Combines Modalities

- Images replaced by <image> tokens in input text.
- Extracted image tokens injected via cross-attention layers.
- Result: a unified representation that allows rich image-text understanding.

Use Cases & Strengths

- Excels at few-shot multimodal tasks.
- Handles variable image counts and diverse input formats.
- Supports inference on both images and videos.
- State-of-the-art performance on VQA, captioning, and more.

Key Takeaways

- Flamingo integrates vision + language with precision and flexibility.
- Leverages strong pretrained models, avoiding retraining from scratch.
- Smart use of attention and gating enables multimodal reasoning.
- Set the stage for models like GPT-4V and Google Gemini.

Availability

- Flamingo, as developed by DeepMind, is not fully open-sourced
- What's Available
  - Model architecture details - Paper
  - Community reimplementations: OpenFlamingo by LAION and others is a replica of Flamingo using open models like CLIP and LLaMA
  - Hugging Face: Some open implementations are partially supported through Transformers + vision encoders.
- What's Not Available
  - Official weights from DeepMind
  - Training data